

LAPORAN PRAKTIKUM

STRUKTUR DATA

“SINGLE LINKED LIST”



disusun Oleh:

NIDA TSABITA ARIBA

2511532025

Dosen Pengampu:

Dr. WAHYUDI, S.T., M.T.

Asisten Praktikum:

JOVANTRI IMMANUEL GULO

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Alhamdulillah rabbil ‘alamin, segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, nikmat, dan hidayah-Nya sehingga saya dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan judul “*SINGLE LINKED LIST*”.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Struktur Data. Laporan ini diharapkan dapat memperdalam wawasan saya mengenai penerapan konsep struktur data, khususnya *QUEUE* dalam bahasa pemrograman Java, serta implementasinya dalam pengelolaan data yang bersifat dinamis.

Saya menyampaikan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Algoritma dan Pemrograman, serta asisten praktikum yang telah memberikan bimbingan dan arahan selama pelaksanaan kegiatan praktikum. Ucapan terima kasih juga saya sampaikan kepada teman-teman praktikan yang telah memberikan bantuan, dukungan, serta kesempatan berdiskusi dalam menyelesaikan tugas praktikum ini.

Saya menyadari bahwa laporan ini masih memiliki banyak kekurangan dan keterbatasan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum di masa mendatang. Meskipun demikian, saya berharap laporan ini dapat memberikan manfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	3
2.1 Program <i>NodeSSL</i>	3
2.2 Program <i>TambahSSL</i>	5
2.3 Program <i>PencarianSSL</i>	14
2.4 Program <i>HapusSSL</i>	20
BAB III KESIMPULAN.....	26
3.1 Kesimpulan	26
DAFTAR PUSTAKA	28

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pembelajaran struktur data, mahasiswa tidak hanya mempelajari konsep dasar pemrograman, tetapi juga mempelajari cara pengelolaan data menggunakan struktur data dinamis. Salah satu struktur data yang penting untuk dipahami adalah *Single Linked List*.

Single Linked List merupakan struktur data linear yang terdiri dari kumpulan *node*, dimana setiap *node* memiliki data dan pointer yang menunjuk ke *node* berikutnya. Struktur data ini bersifat dinamis karena ukuran penyimpanan data dapat bertambah atau berkurang sesuai kebutuhan.

Pada praktikum ini, *Single Linked List* diimplementasikan menggunakan bahasa Java untuk melakukan operasi penambahan *node* pada bagian depan, bagian belakang, dan posisi tertentu. Program juga dapat menampilkan seluruh isi *linked list* secara berurutan.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar *Single Linked List*, penggunaan pointer *next*, serta proses manipulasi *node* dalam struktur data dinamis.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar *Single Linked List* dalam struktur data.
2. Mengetahui cara kerja pointer *next* pada *linked list*.
3. Mempelajari implementasi *Single Linked List* menggunakan Java.
4. Mengimplementasikan operasi penambahan *node* di depan, belakang, dan posisi tertentu.
5. Melatih kemampuan dalam membuat program struktur data dinamis menggunakan *linked list*.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman tentang struktur data *Single Linked List*.
2. Membantu memahami hubungan antar *node* menggunakan pointer.
3. Melatih kemampuan membuat program struktur data dinamis.
4. Membiasakan penggunaan operasi dasar pada *linked list*.
5. Meningkatkan kemampuan logika dalam manipulasi data pada struktur *linked list*.
6. Membantu memahami proses traversal dan penyisipan *node* dalam *linked list*.

BAB II

PEMBAHASAN

2.1 Program *NodeSLL*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan5_2511532025;

public class NodeSLL_2511532025 {
    // node bagian data
    int data;
    // Pointer ke node berikutnya
    NodeSLL_2511532025 next;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532025(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Kode Program 2.1 Program *NodeSLL*

Kode di atas berfungsi untuk membuat sebuah *node* pada struktur data *Single Linked List (SLL)* dalam bahasa Java. Class *NodeSLL_2511532025* digunakan sebagai komponen dasar pembentuk *linked list*, di mana setiap *node* memiliki dua bagian utama, yaitu data dan pointer atau referensi ke *node* berikutnya.

Variabel data digunakan untuk menyimpan nilai atau isi dari *node*, sedangkan variabel *next* digunakan untuk menunjuk ke *node* selanjutnya dalam *linked list*. Jika *node* belum terhubung dengan *node* lain, maka nilai *next* akan bernilai *null*.

Pada *class* ini juga terdapat konstruktor *NodeSLL_2511532025(int data)* yang berfungsi untuk menginisialisasi *node* baru dengan nilai data tertentu. Saat objek *node* dibuat, data akan diisi sesuai parameter yang diberikan, kemudian pointer *next* diatur menjadi *null* karena *node* tersebut belum terhubung ke *node* lain.

Melalui program ini, dapat dipahami bahwa *node* merupakan elemen dasar dalam struktur data *linked list*. Setiap *node* saling

terhubung menggunakan pointer sehingga data dapat disimpan secara dinamis tanpa harus menggunakan ukuran *array* yang tetap.

2.1.1 Langkah Kerja Program

1. Membuat *class NodeSLL_2511532025*

```
public class NodeSLL_2511532025 {
```

Kode Program 2.2 Class *NodeSLL*

Class ini digunakan untuk membuat sebuah *node* pada struktur data *Single Linked List*.

2. Deklarasi variabel *data* dan *next*

```
// node bagian data
int data;
// Pointer ke node berikutnya
NodeSLL_2511532025 next;
```

Kode Program 2.3 Variabel *Data* dan *Next*

- Variabel *data* digunakan untuk menyimpan nilai pada *node*.
- Variabel *next* digunakan sebagai pointer atau penghubung ke *node* berikutnya.

3. Membuat konstruktor *node*

```
// Konstruktor menginisialisasi node dengan data
public NodeSLL_2511532025(int data) {
```

Kode Program 2.4 Konstruktor *Node*

Konstruktor digunakan untuk menginisialisasi objek *node* saat pertama kali dibuat.

4. Mengisi nilai data *node*

```
this.data = data;
```

Kode Program 2.6 *Inisialisasi Data*

Syntax ini digunakan untuk mengisi variabel data dengan nilai yang diberikan melalui parameter.

5. Mengatur pointer *next* menjadi *null*

```
this.next = null;
```

Kode Program 2.7 *Inisialisai Next*

Syntax ini digunakan untuk mengatur agar *node* belum terhubung dengan *node* lain.

2.2 Program *TambahSSL*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan5_2511532025;

public class TambahSSL_2511532025 {
    public static NodeSSL_2511532025 insertAtFront
(NodeSSL_2511532025 head_2025, int value) {
        NodeSSL_2511532025 new_node = new
NodeSSL_2511532025(value);
        new_node.next = head_2025;
        return new_node;
    }
    // fungsi menambahkan node di akhir SLL
    public static NodeSSL_2511532025
insertAtEnd(NodeSSL_2511532025 head_2025, int value) {
        // buat sesudah node dengan sebuah nilai
        NodeSSL_2511532025 newNode = new
NodeSSL_2511532025(value);
        // jika list kosong maka node jadi head
        if (head_2025 == null) {
            return newNode;
        }
        //simpan head ke variabel sementara
        NodeSSL_2511532025 last = head_2025;
        // telusuri ke kode akhir
        while (last.next != null) {
            last = last.next;
        }
        // ubah pointer
        last.next = newNode;
        return head_2025;
    }
    static NodeSSL_2511532025 GetNode(int data) {
        return new NodeSSL_2511532025(data);
    }
}
```

```

        static NodeSLL_2511532025 insertPos(NodeSLL_2511532025
headNode, int position, int value) {
    NodeSLL_2511532025 head = headNode;
    if (position < 1)
        System.out.print("Invalid position");
    if (position == 1) {
        NodeSLL_2511532025 new_node = new
NodeSLL_2511532025(value);
        new_node.next = head;
        return new_node;
    } else {
        while (position -- != 0) {
            if (position == 1) {
                NodeSLL_2511532025 newNode =
GetNode(value);
                newNode.next = headNode.next;
                headNode.next = newNode;
                break;
            }
            headNode = headNode.next;
        }
        if (position != 1)
            System.out.print("Posisi di luar
jangkauan");
    }
    return head;
}

public static void printList (NodeSLL_2511532025
head) {
    NodeSLL_2511532025 curr_2025 = head;
    while (curr_2025.next != null) {
        System.out.print(curr_2025.data + "-->");
        curr_2025 = curr_2025.next;
    }
    if (curr_2025.next == null) {
        System.out.print(curr_2025.data);
    }

    System.out.println();
}

public static void main(String[] args) {
    // buat linked list 2->3->4->5->
    NodeSLL_2511532025 head_2025 = new
NodeSLL_2511532025(2);
    head_2025.next = new NodeSLL_2511532025(3);
    head_2025.next.next = new
NodeSLL_2511532025(5);
    head_2025.next.next.next = new
NodeSLL_2511532025(6);
    // cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList(head_2025);
    //tambahkan node baru di depan
    System.out.print("tambah 1 simpul di depan:
");

    int data_2025 = 1;
    head_2025 = insertAtFront(head_2025,
data_2025);
}

```

```

        // cetak update list
        printList(head_2025);
        // tambahkan node baru di belakang
        System.out.print("tambah 1 simpul di
belakang: ");
        int data2_2025 = 7;
        head_2025 = insertAtEnd(head_2025,
data2_2025);
        // cetak update list
        printList(head_2025);
        System.out.print("tambah 1 simpul ke data 4:
");
        int data3_2025 = 4;
        int pos_2025 = 4;
        head_2025 = insertPos(head_2025, pos_2025,
data3_2025);
        // cetak update list
        printList(head_2025);
    }
}

```

Kode Program 2.8 Program *TambahSLL*

Kode di atas digunakan untuk melakukan operasi penambahan *node* pada struktur data *Single Linked List*. Program ini dapat menambahkan *node* di bagian depan *linked list*, di bagian akhir *linked list*, dan pada posisi tertentu. Selain itu, program juga dapat menampilkan seluruh isi *linked list* menggunakan *method printList()*.

Program dimulai dengan membuat *linked list* awal berisi data 2 -> 3 -> 5 -> 6. Selanjutnya dilakukan beberapa operasi penambahan *node*, yaitu menambahkan data 1 di depan *linked list*, menambahkan data 7 di belakang *linked list*, dan menambahkan data 4 pada posisi tertentu sehingga *linked list* menjadi lebih lengkap dan teratur.

Melalui program ini dapat dipahami cara kerja *Single Linked List*, terutama dalam proses manipulasi *node* menggunakan pointer *next*.

2.2.1 Langkah Kerja Program

1. Membuat *class TambahSLL_2511532025*

```
public class TambahSLL_2511532025 {
```

Kode Program 2.9 *Class TambahSLL*

Class ini digunakan untuk melakukan operasi penambahan *node* pada struktur data *Single Linked List*.

2. Membuat *method insertAtFront()*

```
public static NodeSLL_2511532025 insertAtFront  
(NodeSLL_2511532025 head_2025, int value) {
```

Kode Program 2.10 *Method InsertAtFront*

Method ini digunakan untuk menambahkan *node* baru di bagian depan *linked list*.

3. Membuat *node baru*

```
NodeSLL_2511532025 new_node = new  
NodeSLL_2511532025(value);
```

Kode Program 2.11 Pembuatan *Node Baru*

Digunakan untuk membuat *node* baru dengan nilai data yang diberikan.

4. Menghubungkan *node* baru ke *head*

```
new_node.next = head_2025;
```

Kode Program 2.12 Menghubungkan *Node*

Digunakan untuk menghubungkan *node* baru ke *linked list* sebelumnya.

5. Mengembalikan *node* baru sebagai *head*

```
return new_node;  
}
```

Kode Program 2.13 *Return Head Baru*

Digunakan untuk menjadikan *node* baru sebagai *head* linked list.

6. Membuat *method insertAtEnd()*

```
// fungsi menambahkan node di akhir SLL
public static NodeSLL_2511532025
insertAtEnd(NodeSLL_2511532025 head_2025, int value) {
    // buat sesudah node dengan sebuah nilai
    NodeSLL_2511532025 newNode = new
NodeSLL_2511532025(value);
```

Kode Program 2.14 *Method InsertAtEnd*

Method ini digunakan untuk menambahkan *node* di bagian akhir *linked list*.

7. Mengecek apakah *linked list* kosong

```
// jika list kosong maka node jadi head
if (head_2025 == null) {
    return newNode;
}
//simpan head ke variabel sementara
NodeSLL_2511532025 last = head_2025;
// telusuri ke kode akhir
```

Kode Program 2.15 Pengecekan *Linked List* Kosong

Digunakan untuk menjadikan *node* baru sebagai *head* jika *linked list* masih kosong

8. Menelusuri *node* sampai akhir

```
while (last.next != null) {
    last = last.next;
}
```

Kode Program 2.16 Penelusuran *Node*

Digunakan untuk mencari *node* terakhir pada *linked list*.

9. Menghubungkan *node* terakhir dengan *node* baru

```
// ubah pointer
last.next = newNode;
```

```

        return head_2025;
    }
    static NodeSLL_2511532025 GetNode(int data) {
        return new NodeSLL_2511532025(data);
    }
}

```

Kode Program 2.17 Menambahkan *Node* di Akhir

Digunakan untuk menghubungkan *node* terakhir dengan *node* baru.

10. Membuat *method insertPos()*

```

    static NodeSLL_2511532025
    insertPos(NodeSLL_2511532025 headNode, int position,
    int value) {
        NodeSLL_2511532025 head = headNode;
    }

```

Kode Program 2.18 *Method InsertPos*

Method ini digunakan untuk menambahkan *node* pada posisi tertentu.

11. Mengecek posisi *node*

```

    if (position < 1)
        System.out.print("Invalid position");

```

Kode Program 2.19 Pengecekan Posisi

Digunakan untuk mengecek apakah posisi yang dimasukkan valid.

12. Menambahkan *node* pada posisi pertama

```

    if (position == 1) {
        NodeSLL_2511532025 new_node = new
NodeSLL_2511532025(value);
        new_node.next = head;
        return new_node;
    } else {
        while (position -- != 0) {
            if (position == 1) {
                NodeSLL_2511532025 newNode
= GetNode(value);

```

Kode Program 2.20 *Insert* Posisi Pertama

Digunakan untuk menambahkan *node* di posisi pertama *linked list*.

13. Menambahkan *node* pada posisi tertentu

```
newNode.next =  
headNode.next;  
headNode.next = newNode;  
break;  
}  
headNode = headNode.next;  
}  
if (position != 1)  
    System.out.print("Posisi di luar  
jangkauan");  
}  
return head;  
}
```

Kode Program 2.21 *Insert* Posisi Tertentu

Digunakan untuk menyisipkan *node* baru di antara *node linked list*.

14. Membuat *method printList()*

```
public static void printList  
(NodeSLL_2511532025 head) {  
    NodeSLL_2511532025 curr_2025 = head;  
    while (curr_2025.next != null) {
```

Kode Program 2.22 *Method PrintList()*

Method ini digunakan untuk menampilkan seluruh isi *linked list*.

15. Menampilkan data *linked list*

```
System.out.print(curr_2025.data + "--  
>");  
curr_2025 = curr_2025.next;  
}  
if (curr_2025.next == null) {
```

```

System.out.print(curr_2025.data);
    }
    System.out.println();
}

```

Kode Program 2.23 Menampilkan data

Digunakan untuk mencetak data *node* secara berurutan.

16. Membuat *method main()*

```

public static NodeSLL_2511532025 insertAtFront
(NodeSLL_2511532025 head_2025, int value) {

```

Kode Program 2.24 *Method Main*

Method utama yang digunakan untuk menjalankan program.

17. Membuat *linked list* awal

```

NodeSLL_2511532025 new_node = new
NodeSLL_2511532025(value);

```

Kode Program 2.25 Pembuatan *Linked List*

Digunakan untuk membuat *linked list* awal dengan data 2 -> 3
-> 5 -> 6.

18. Menampilkan *linked list* awal

```

new_node.next = head_2025;

```

Kode Program 2.26 Menampilkan *Linked List* Awal

Digunakan untuk menampilkan isi *linked list* sebelum dilakukan penambahan *node*.

19. Menambahkan *node* di depan

```

return new_node;
}

```

Kode Program 2.27 *Insert* Depan

Digunakan untuk menambahkan *node* baru di bagian depan *linked list*.

20. Menambahkan *node* di belakang

```
// fungsi menambahkan node di akhir SLL
public static NodeSLL_2511532025
insertAtEnd(NodeSLL_2511532025 head_2025, int value) {
    // buat sesudah node dengan sebuah nilai
    NodeSLL_2511532025 newNode = new
NodeSLL_2511532025(value);
```

Kode Program 2.28 *Insert* Belakang

Digunakan untuk menambahkan *node* baru di bagian akhir *linked list*.

21. Menambahkan *node* pada posisi tertentu

```
// jika list kosong maka node jadi head
if (head_2025 == null) {
    return newNode;
}
//simpan head ke variabel sementara
NodeSLL_2511532025 last = head_2025;
// telusuri ke kode akhir
```

Kode Program 2.29 *Insert* Posisi Tertentu

Digunakan untuk menambahkan *node* baru pada posisi tertentu dalam *linked list*.

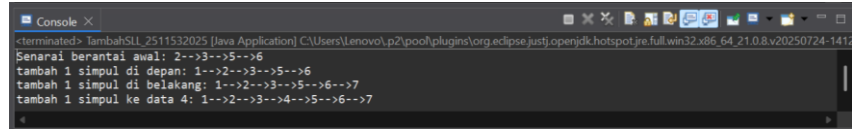
22. Menampilkan hasil akhir *linked list*

```
while (last.next != null) {
    last = last.next;
}
```

Kode Program 2.30 Menampilkan Hasil Akhir

Digunakan untuk menampilkan *linked list* setelah dilakukan proses penambahan *node*.

2.2.2 Hasil Program



```
<terminated> TambahSLL_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.21.0.8.v20250724-1412
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 2.1 Hasil Program *TambahSLL*

Berdasarkan hasil eksekusi program, berhasil ditambahkan ke dalam *Single Linked List* pada berbagai posisi. Awalnya *linked list* berisi data:

$2 \rightarrow 3 \rightarrow 5 \rightarrow 6$

Kemudian dilakukan penambahan *node* di depan dengan nilai 1 sehingga list berubah menjadi:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6$

Selanjutnya program menambahkan *node* di belakang dengan nilai 7 sehingga *linked list* menjadi:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7$

Terakhir, program menambahkan *node* bernilai 4 pada posisi tertentu sehingga hasil akhir *linked list* adalah:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

Hasil tersebut menunjukkan bahwa operasi penambahan *node* pada *Single Linked List* berhasil dilakukan dengan baik, baik di bagian depan, belakang, maupun posisi tertentu. Program ini juga memperlihatkan bagaimana pointer *next* pada setiap *node* diubah agar hubungan antar *node* tetap tersusun dengan benar.

2.3 Program *PencarianSLL*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan5_2511532025;
```

```

public class PencarianSLL_2511532025 {
    static boolean searchKey(NodeSLL_2511532025 head_2025, int
key_2025) {
        NodeSLL_2511532025 curr_2025 = head_2025;
        while (curr_2025 != null) {
            if (curr_2025.data == key_2025)
                return true;
            curr_2025 = curr_2025.next;
        }
        return false;
    }
    public static void traversal (NodeSLL_2511532025 head_2025) {
        //mulai dari head
        NodeSLL_2511532025 curr_2025 = head_2025;
        //telusuri sampai pointer null
        while (curr_2025 != null) {
            System.out.print(" " + curr_2025.data);
            curr_2025 = curr_2025.next;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        NodeSLL_2511532025 head_2025 = new
NodeSLL_2511532025(14);
        head_2025.next = new NodeSLL_2511532025(21);
        head_2025.next.next = new NodeSLL_2511532025(13);
        head_2025.next.next.next = new
NodeSLL_2511532025(30);
        head_2025.next.next.next.next = new
NodeSLL_2511532025(10);
        System.out.print("Penelusuran SLL : ");
        traversal(head_2025);
        // data yang akan dicari
        int key_2025 = 30;
        System.out.print("cari data " +key_2025+ " = ");
        if (searchKey(head_2025, key_2025))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}

```

Kode Program 2.31 Program *PencarianSLL*

Kode program di atas berfungsi untuk menunjukkan proses pencarian data pada struktur data Singly Linked List (SLL) menggunakan bahasa Java. Program ini melakukan traversal atau penelusuran seluruh node pada linked list, kemudian mencari apakah suatu data tertentu terdapat di dalam linked list tersebut.

Program menggunakan konsep singly linked list, yaitu kumpulan node yang saling terhubung menggunakan pointer next. Setiap node menyimpan data dan referensi menuju node berikutnya. Pencarian data dilakukan secara linear, yaitu menelusuri node satu per satu mulai dari head hingga data ditemukan atau hingga mencapai node terakhir.

Melalui program ini dapat dipahami bagaimana cara kerja proses traversal dan searching pada singly linked list menggunakan method `traversal()` dan `searchKey()`.

2.3.1 Langkah Kerja Program *ReverseData*

1. Membuat *class* `PencarianSLL_2511532025`

```
public class PencarianSLL_2511532025 {
```

Kode Program 2.32 *Class* `PencarianSLL`

Class ini digunakan untuk mengelola proses pencarian data pada *single linked list*.

2. *Method* `searchKey()`

```
static boolean searchKey(NodeSLL_2511532025
head_2025, int key_2025) {
    NodeSLL_2511532025 curr_2025 = head_2025;
    while (curr_2025 != null) {
        if (curr_2025.data == key_2025)
            return true;
        curr_2025 = curr_2025.next;}
    return false;
}
```

Kode Program 2.33 *Method* `searchKey`

Method ini digunakan untuk mencari data tertentu pada linked list.

Langkah kerja method:

- Menyimpan head ke variabel curr
- Menelusuri linked list satu per satu
- Membandingkan data node dengan key yang dicari
- Jika data ditemukan maka method mengembalikan nilai true
- Jika data tidak ditemukan hingga akhir list maka mengembalikan false

3. *Method traversal()*

```
public static void traversal (NodeSLL_2511532025
head_2025) {
    //mulai dari head
    NodeSLL_2511532025 curr_2025 = head_2025;
    //telusuri sampai pointer null
    while (curr_2025 != null) {
        System.out.print(" " + curr_2025.data);
        curr_2025 = curr_2025.next;
    }
    System.out.println();
}
```

Kode Program 2.34 *Method Traversal*

Method traversal digunakan untuk menampilkan seluruh isi linked list dari node pertama hingga node terakhir.

Langkah kerja method:

- Memulai traversal dari head
- Menampilkan data setiap node
- Berpindah ke node berikutnya menggunakan pointer next
- Berhenti ketika curr bernilai null

4. Membuat *method main()*

```
public static void main(String[] args) {
```

Kode Program 2.35 *Method Main*

Method main digunakan sebagai program utama untuk menjalankan proses traversal dan pencarian data pada linked list.

5. Membuat *linked list*

```
NodeSLL_2511532025 head_2025 = new
NodeSLL_2511532025(14);
    head_2025.next = new NodeSLL_2511532025(21);
    head_2025.next.next = new
NodeSLL_2511532025(13);
    head_2025.next.next.next = new
NodeSLL_2511532025(30);
    head_2025.next.next.next.next = new
NodeSLL_2511532025(10);
```

Kode Program 2.36 *Return Head Baru*

Baris program ini digunakan untuk membuat single linked list.

6. Menampilkan isi *linked list*

```
System.out.print("Penelusuran SLL : ");
traversal(head_2025);
```

Kode Program 2.37 Menampilkan Traversal

Program menampilkan seluruh isi linked list menggunakan method traversal().

7. Menentukan data yang dicari

```
// data yang akan dicari
int key_2025 = 30;
System.out.print("cari data " +key_2025+ " =
");
```

Kode Program 2.38 Menentukan Key

Variabel `key_2025` digunakan untuk menyimpan data yang akan dicari pada linked list.

8. Melakukan proses pencarian data

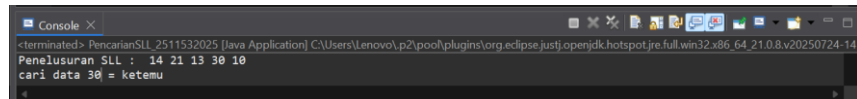
```
if (searchKey(head_2025, key_2025))
    System.out.println("ketemu");
else
    System.out.println("tidak ada");
}
```

Kode Program 2.39 Proses *Searching*

Program memanggil method `searchKey()` untuk mencari data 30 pada linked list.

- Jika data ditemukan maka tampil tulisan “ketemu”
- Jika data tidak ditemukan maka tampil tulisan “tidak ada”

2.3.2 Hasil Program



Gambar 2.3 Hasil Program *PencarianSLL*

Berdasarkan hasil eksekusi program, linked list berhasil ditelusuri menggunakan method `traversal()`, sehingga seluruh data dapat ditampilkan secara berurutan yaitu:

14 → 21 → 13 → 30 → 10

Selanjutnya program melakukan pencarian data dengan nilai 30 menggunakan method `searchKey()`. Proses pencarian dilakukan secara linear dengan menelusuri node satu per satu mulai dari head hingga data ditemukan.

Karena data 30 terdapat di dalam linked list, maka program menampilkan output:

ketemu

Hasil tersebut menunjukkan bahwa proses searching pada singly linked list berhasil dilakukan dengan baik menggunakan metode traversal linear. Program ini juga memperlihatkan bagaimana setiap node diperiksa satu per satu hingga data yang dicari ditemukan atau mencapai akhir linked list.

2.4 Program *HapusSLL*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan5_2511532025;

public class HapusSLL_2511532025 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511532025
deleteHead_2025(NodeSLL_2511532025 head_2025)
    {
        // jika SLL kosong
        if (head_2025 == null)
            return null;
        // pindahkan head ke node berikutnya
        head_2025 = head_2025.next;
        // return head baru
        return head_2025;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532025
removeLastNode_2025(NodeSLL_2511532025 head_2025) {
        // jika list kosong, return null
        if (head_2025 == null) {
            return null;
        }
        // jika list satu node, hapus node dan return
        null

        if (head_2025.next == null) {
            return null;
        }
        // temukan node terakhir ke dua
        NodeSLL_2511532025 secondLast = head_2025;
        while (secondLast.next.next != null) {
            secondLast = secondLast.next;
        }
        // hapus node terakhir
        secondLast.next = null;
        return head_2025;
    }

    public static NodeSLL_2511532025
deleteNode_2025(NodeSLL_2511532025 head_2025, int position) {
        NodeSLL_2511532025 temp_2025 = head_2025;
```

```

        NodeSLL_2511532025 prev_2025 = null;
        // jika linked list null
        if (temp_2025 == null)
            return head_2025;
        // kasus 1: head dihapus
        if (position == 1) {
            head_2025 = temp_2025.next;
            return head_2025;
        }
        // kasus 2: menghapus node ditengah
        // telusuri node yang dihapus
        for (int i_2025 = 1; temp_2025 != null && i_2025 <
position; i_2025++) {
            prev_2025 = temp_2025;
            temp_2025 = temp_2025.next;
        }
        // jika ditemukan, hapus node
        if (temp_2025 != null) {
            prev_2025.next = temp_2025.next;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_2025;
    }

    // fungsi mencetak SLL
    public static void printList_2025(NodeSLL_2511532025
head_2025) {
        NodeSLL_2511532025 curr_2025 = head_2025;
        while (curr_2025.next != null) {
            System.out.print(curr_2025.data + "-->");
            curr_2025 = curr_2025.next;
        }
        if (curr_2025.next == null) {
            System.out.println(curr_2025.data);
        }
        System.out.println();
    }

    // kelas main
    public static void main(String[] args) {
        // buat SLL 1->2->3->4->5->6->null
        NodeSLL_2511532025 head_2025 = new
NodeSLL_2511532025(1);
        head_2025.next = new NodeSLL_2511532025(2);
        head_2025.next.next = new NodeSLL_2511532025(3);
        head_2025.next.next.next = new
NodeSLL_2511532025(4);
        head_2025.next.next.next.next = new
NodeSLL_2511532025(5);
        head_2025.next.next.next.next.next = new
NodeSLL_2511532025(6);
        // cetak list awal
        System.out.println("list awal: ");
        printList_2025(head_2025);
        // hapus head
        head_2025 = deleteHead_2025(head_2025);
        System.out.println("Liat setelah head dihapus: ");
    }

```

```

        printList_2025(head_2025);
        // hapus node terakhir
        head_2025 = removeLastNode_2025(head_2025);
        System.out.println("List setelah simpul terakhir di
hapus: ");
        printList_2025(head_2025);
        //deleting node at position 2
        int position = 2;
        head_2025 = deleteNode_2025(head_2025, position);
        // print list after deletion
        System.out.println("List setelah posisi 2 dihapus:
");
        printList_2025(head_2025);
    }
}

```

Kode Program 2.40 Program *HapusSLL*

Kode program di atas berfungsi untuk menunjukkan proses penghapusan *node* pada struktur data *Single Linked List* (SLL) menggunakan bahasa Java. Program ini menyediakan beberapa *method* untuk menghapus *node* di bagian depan (*head*), *node* terakhir, dan *node* pada posisi tertentu di dalam *linked list*.

Program menggunakan konsep *Single Linked List*, yaitu struktur data linear yang terdiri dari kumpulan *node* yang saling terhubung melalui pointer *next*. Setiap *node* menyimpan data dan referensi ke *node* berikutnya. Dengan struktur ini, proses penghapusan data dilakukan dengan mengubah hubungan antar *node* tanpa harus menggeser elemen seperti pada *array*.

Melalui program ini dapat dipahami bagaimana cara kerja operasi penghapusan *node* pada *linked list*, mulai dari menghapus *node* pertama, *node* terakhir, hingga *node* pada posisi tertentu. Program juga menampilkan isi *linked list* sebelum dan sesudah proses penghapusan dilakukan.

2.4.1 Langkah Kerja Program

Program diawali dengan pembuatan class `HapusSLL_2511532025` yang digunakan untuk mengelola operasi penghapusan node pada struktur data Single Linked List (SLL). Di dalam class ini terdapat beberapa method yang berfungsi untuk menghapus node pada posisi tertentu serta menampilkan isi linked list.

Langkah berikutnya adalah membuat method `deleteHead_2025()`. Method ini digunakan untuk menghapus node paling depan atau head pada linked list. Program terlebih dahulu mengecek apakah linked list kosong. Jika kosong maka method akan mengembalikan nilai null. Namun jika linked list berisi data, maka posisi head dipindahkan ke node berikutnya sehingga node pertama terhapus dari linked list.

Selanjutnya dibuat method `removeLastNode_2025()` yang digunakan untuk menghapus node terakhir pada linked list. Program akan memeriksa apakah linked list kosong atau hanya memiliki satu node. Jika linked list hanya memiliki satu node maka list akan menjadi kosong. Jika terdapat lebih dari satu node, program akan menelusuri linked list hingga menemukan node kedua terakhir, kemudian pointer next pada node tersebut diubah menjadi null sehingga node terakhir terhapus.

Program juga memiliki method `deleteNode_2025()` yang digunakan untuk menghapus node pada posisi tertentu. Method ini menggunakan variabel sementara untuk menelusuri linked list satu per satu hingga mencapai posisi node yang akan dihapus. Setelah node ditemukan, pointer dari node sebelumnya akan dihubungkan ke node setelah node

yang dihapus sehingga node tersebut tidak lagi menjadi bagian dari linked list.

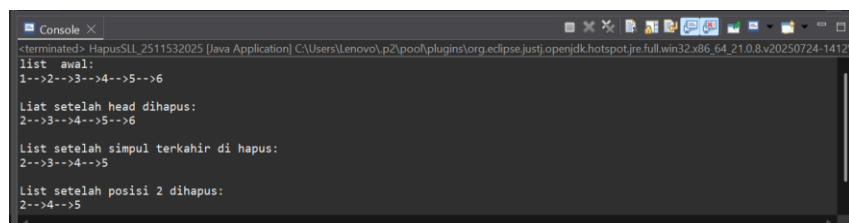
Selain itu, dibuat method `printList_2025()` yang digunakan untuk menampilkan seluruh isi linked list. Method ini melakukan traversal mulai dari head hingga node terakhir dengan menampilkan data pada setiap node secara berurutan.

Pada method `main()`, program pertama kali membuat linked list awal yang berisi data $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$. Setelah itu program menampilkan isi linked list awal menggunakan method `printList_2025()`.

Kemudian program melakukan penghapusan node pertama menggunakan method `deleteHead_2025()` sehingga data 1 terhapus dan linked list berubah menjadi $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$. Setelah itu program menghapus node terakhir menggunakan method `removeLastNode_2025()` sehingga data 6 terhapus dan linked list menjadi $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

Langkah terakhir adalah menghapus node pada posisi ke-2 menggunakan method `deleteNode_2025()`. Pada proses ini node dengan data 3 dihapus sehingga hasil akhir linked list menjadi $2 \rightarrow 4 \rightarrow 5$. Setelah seluruh proses selesai dilakukan, program kembali menampilkan isi linked list untuk menunjukkan hasil akhir setelah penghapusan data.

2.4.2 Hasil Program



```
Console X
-terminated> HapusSLL_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\
list awal:
1-->2-->3-->4-->5-->6

List setelah head dihapus:
2-->3-->4-->5-->6

List setelah simpul terakhir di hapus:
2-->3-->4-->5

List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.3 Hasil Program *HapusSLL*

Berdasarkan hasil eksekusi program, operasi penghapusan *node* pada *Single Linked List* berhasil dilakukan dengan baik. Awalnya *linked list* berisi data:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$$

Kemudian program menghapus *node* pertama (*head*) sehingga *linked list* berubah menjadi:

$$2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$$

Selanjutnya dilakukan penghapusan *node* terakhir, yaitu *node* dengan nilai 6, sehingga list menjadi:

$$2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Terakhir, program menghapus *node* pada posisi ke-2 sehingga data 3 dihapus dan hasil akhir *linked list* menjadi:

$$2 \rightarrow 4 \rightarrow 5$$

Hasil tersebut menunjukkan bahwa proses penghapusan *node* pada *Single Linked List* dapat dilakukan pada berbagai posisi dengan cara mengubah pointer *next* antar *node*. Program ini juga memperlihatkan bagaimana *linked list* tetap terhubung dengan benar setelah proses penghapusan dilakukan.

BAB III

KESIMPULAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Singly Linked List (SLL) merupakan salah satu struktur data linear dalam bahasa Java yang digunakan untuk menyimpan dan mengelola data secara dinamis menggunakan hubungan antar node. Setiap node terdiri dari data dan pointer yang menunjuk ke node berikutnya, sehingga data dapat saling terhubung membentuk sebuah linked list.

Melalui beberapa program yang telah dibuat, seperti class `NodeSLL_2511532025`, program penambahan node (`TambahSLL`), penghapusan node (`HapusSLL`), dan pencarian data (`PencarianSLL`), dapat dipahami bahwa linked list memiliki fleksibilitas dalam pengelolaan data dibandingkan array karena ukuran data dapat berubah secara dinamis tanpa harus menentukan kapasitas di awal program.

Class `NodeSLL_2511532025` berfungsi sebagai dasar pembentukan node dalam linked list, dimana setiap node menyimpan data dan referensi ke node berikutnya. Dengan adanya class node ini, proses pengelolaan linked list dapat dilakukan dengan lebih mudah melalui manipulasi pointer `next`.

Pada program `TambahSLL`, operasi penambahan node dapat dilakukan di berbagai posisi, yaitu di depan, di belakang, maupun pada posisi tertentu di dalam linked list. Program ini menunjukkan bagaimana pointer antar node diubah agar node baru dapat terhubung dengan benar tanpa merusak struktur linked list.

Selanjutnya, program `HapusSLL` menunjukkan proses penghapusan node pada linked list, baik pada bagian head, node terakhir, maupun posisi tertentu. Proses penghapusan dilakukan dengan mengubah

hubungan antar node sehingga node yang dihapus tidak lagi terhubung dalam linked list.

Selain itu, program PencarianSLL memperlihatkan proses traversal dan searching data pada linked list. Pencarian dilakukan secara linear dengan menelusuri node satu per satu mulai dari head hingga data ditemukan atau mencapai akhir linked list.

Dengan demikian, praktikum ini membantu memahami konsep dasar, cara kerja, serta implementasi struktur data Singly Linked List dalam Java. Praktikum ini juga meningkatkan kemampuan dalam membuat program untuk menambah, menghapus, menelusuri, dan mencari data secara lebih fleksibel, efisien, dan terstruktur.

DAFTAR PUSTAKA

Wahyudi, S.T., M.T., *Struktur Data: Pekan 6 Linked List Single/Double*. Padang: Universitas Andalas, 2025.

Tugas Praktikum Pekan 5: Struktur Data

Topik : Implementasi *Single Linked List*

Tema : Simulasi Antrian Rumah Sakit

1. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep *Single Linked List* yang bekerja secara dinamis menggunakan pointer (referensi antar *node*). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah *node* yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (*FIFO — First In, First Out*).

2. Kode Program

1) Kelas ADT Pasien

```
package pekan5_2511532025;

public class Pasien_2511532025 {

    // atribut data pasien
    String namaPasien_2025;
    String penyakit_2025;
    int nomorAntrian_2025;

    // pointer ke pasien berikutnya
    Pasien_2511532025 next_2025;

    // konstruktor untuk menginisialisasi data pasien
    public Pasien_2511532025(String namaPasien_2025, String
    penyakit_2025, int nomorAntrian_2025) {

        this.namaPasien_2025 = namaPasien_2025;
        this.penakit_2025 = penyakit_2025;
        this.nomorAntrian_2025 = nomorAntrian_2025;

        // node berikutnya masih kosong
        this.next_2025 = null;
    }

    // getter nama pasien
    public String getNamaPasien_2025() {
        return namaPasien_2025;
    }
}
```

```

// setter nama pasien
public void setNamaPasien_2025(String namaPasien_2025) {
    this.namaPasien_2025 = namaPasien_2025;
}

// getter penyakit
public String getPenyakit_2025() {
    return penyakit_2025;
}

// setter penyakit
public void setPenyakit_2025(String penyakit_2025) {
    this.penakit_2025 = penyakit_2025;
}

// getter nomor antrian
public int getNomorAntrian_2025() {
    return nomorAntrian_2025;
}

// setter nomor antrian
public void setNomorAntrian_2025(int nomorAntrian_2025) {
    this.nomorAntrian_2025 = nomorAntrian_2025;
}

// getter pointer next
public Pasien_2511532025 getNext_2025() {
    return next_2025;
}

// setter pointer next
public void setNext_2025(Pasien_2511532025 next_2025) {
    this.next_2025 = next_2025;
}
}

```

2) Kelas Driver

```

package pekan5_2511532025;

import java.util.Scanner;

// class driver rumah sakit
public class RumahSakit_2511532025 {

    // head linked list
    static Pasien_2511532025 head_2025 = null;

    // counter nomor antrian
    static int counter_2025 = 0;

    // method menambahkan pasien di akhir linked list
    public static void daftarkanPasien_2025(String
nama_2025,String penyakit_2025) {

        // nomor antrian bertambah otomatis
    }
}

```

```

        counter_2025++;

        // membuat node pasien baru
        Pasien_2511532025 pasienBaru_2025 = new
Pasien_2511532025(nama_2025, penyakit_2025, counter_2025);

        // jika linked list kosong
        if (head_2025 == null) {
            head_2025 = pasienBaru_2025;
        } else {

            // traversal ke node terakhir
            Pasien_2511532025 temp_2025 = head_2025;

            while (temp_2025.next_2025 != null) {
                temp_2025 = temp_2025.next_2025;
            }

            // hubungkan node terakhir ke pasien baru
            temp_2025.next_2025 = pasienBaru_2025;
        }

        System.out.println("Pasien berhasil didaftarkan!
Nomor Antrian: " + counter_2025);
    }

    // method memanggil pasien pertama
    public static void panggilPasien_2025() {

        // cek apakah antrian kosong
        if (head_2025 == null) {
            System.out.println("Antrian kosong");
            return;
        }

        // tampilkan pasien yang dipanggil
        System.out.println("\nPasien dipanggil:");
        System.out.println("Nomor Antrian : " +
head_2025.nomorAntrian_2025);
        System.out.println("Nama Pasien : " +
head_2025.namaPasien_2025);
        System.out.println("Keluhan : " +
head_2025.penyakit_2025);

        // hapus head
        head_2025 = head_2025.next_2025;
    }

    // method menampilkan seluruh antrian
    public static void tampilkanAntrian_2025() {

        // cek linked list kosong
        if (head_2025 == null) {
            System.out.println("Antrian kosong");
            return;
        }
    }

```

```

// traversal linked list
Pasien_2511532025 temp_2025 = head_2025;

System.out.println("\n=== Daftar Antrian Pasien
===");

while (temp_2025 != null) {

    System.out.println("No Antrian : " +
temp_2025.nomorAntrian_2025);
    System.out.println("Nama      : " +
temp_2025.namaPasien_2025);
    System.out.println("Keluhan   : " +
temp_2025.penyakit_2025);
    System.out.println("-----");

    temp_2025 = temp_2025.next_2025;
}

// method mencari pasien berdasarkan nama
public static void cariPasien_2025(String
namaCari_2025) {

    // cek linked list kosong
    if (head_2025 == null) {
        System.out.println("Antrian kosong");
        return;
    }

    Pasien_2511532025 temp_2025 = head_2025;

    boolean ditemukan_2025 = false;

    while (temp_2025 != null) {

        // pencarian case insensitive
        if
(temp_2025.namaPasien_2025.equalsIgnoreCase(namaCari_2025))
{

            System.out.println("\nPasien ditemukan");
            System.out.println("Nomor Antrian : " +
temp_2025.nomorAntrian_2025);
            System.out.println("Nama Pasien  : " +
temp_2025.namaPasien_2025);
            System.out.println("Keluhan     : " +
temp_2025.penyakit_2025);

            ditemukan_2025 = true;
            break;
        }

        temp_2025 = temp_2025.next_2025;
    }

    // jika pasien tidak ditemukan

```

```

        if (!ditemukan_2025) {
            System.out.println("Pasien tidak ditemukan");
        }
    }

    // method cek status antrian
    public static void cekStatusAntrian_2025() {

        // cek linked list kosong
        if (head_2025 == null) {
            System.out.println("Antrian kosong");
            return;
        }

        int jumlah_2025 = 0;

        Pasien_2511532025 temp_2025 = head_2025;

        // menghitung jumlah pasien
        while (temp_2025 != null) {
            jumlah_2025++;
            temp_2025 = temp_2025.next_2025;
        }

        System.out.println("\n=== Status Antrian ===");
        System.out.println("Jumlah Pasien : " +
jumlah_2025);
        System.out.println("Pasien Terdepan : " +
head_2025.namaPasien_2025);
    }

    // method utama program
    public static void main(String[] args) {
        Scanner input_2025 = new Scanner(System.in);

        int pilihan_2025;

        do {

            // menu program
            System.out.println("\n=== Antrian Rumah Sakit
NIM: 2511532025 ===");
            System.out.println("1. Daftarkan Pasien");
            System.out.println("2. Panggil Pasien");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Cari Pasien");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");

            System.out.print("Pilihan : ");
            pilihan_2025 = input_2025.nextInt();
            input_2025.nextLine();

            switch (pilihan_2025) {

                case 1:
                    // input data pasien

```

```

        System.out.print("Masukkan Nama Pasien
: ");
        String nama_2025 =
input_2025.nextLine();

        System.out.print("Masukkan Keluhan :
");
        String keluhan_2025 =
input_2025.nextLine();

        // memanggil method daftar pasien
        daftarkanPasien_2025(nama_2025,
keluhan_2025);

        break;

    case 2:
        // memanggil pasien
        panggilPasien_2025();

        break;

    case 3:
        // menampilkan antrian
        tampilkanAntrian_2025();

        break;

    case 4:
        // input nama pasien
        System.out.print("Masukkan nama pasien
yang dicari : ");

        String cari_2025 =
input_2025.nextLine();

        // mencari pasien
        cariPasien_2025(cari_2025);

        break;

    case 5:
        // cek status antrian
        cekStatusAntrian_2025();

        break;

    case 6:
        System.out.println("Program selesai");
        break;

    default:

        System.out.println("Pilihan tidak
valid");
}

```

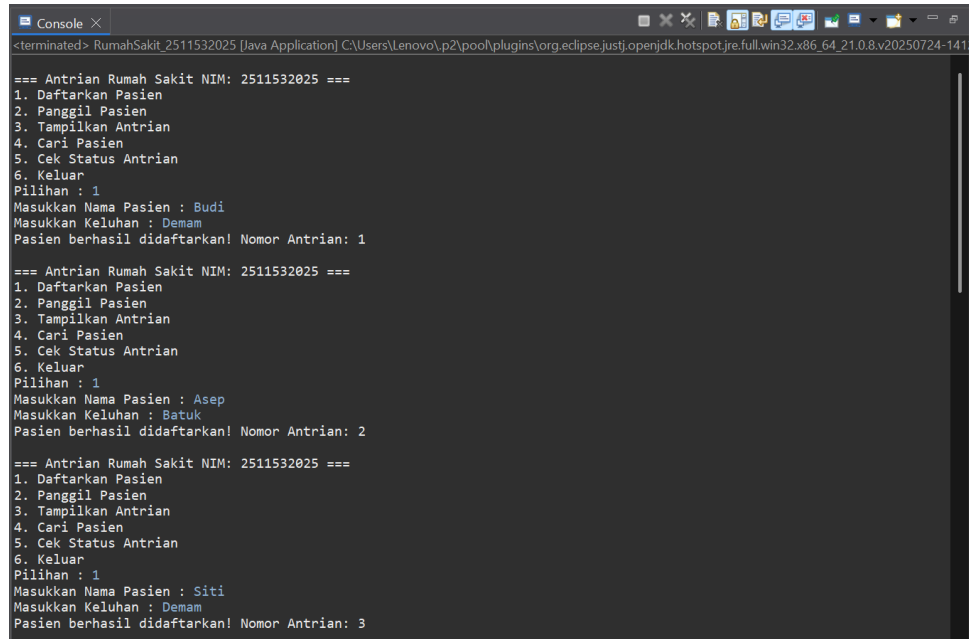
```

    } while (pilihan_2025 != 6);

    input_2025.close();
}
}

```

3. Tampilan Output Program



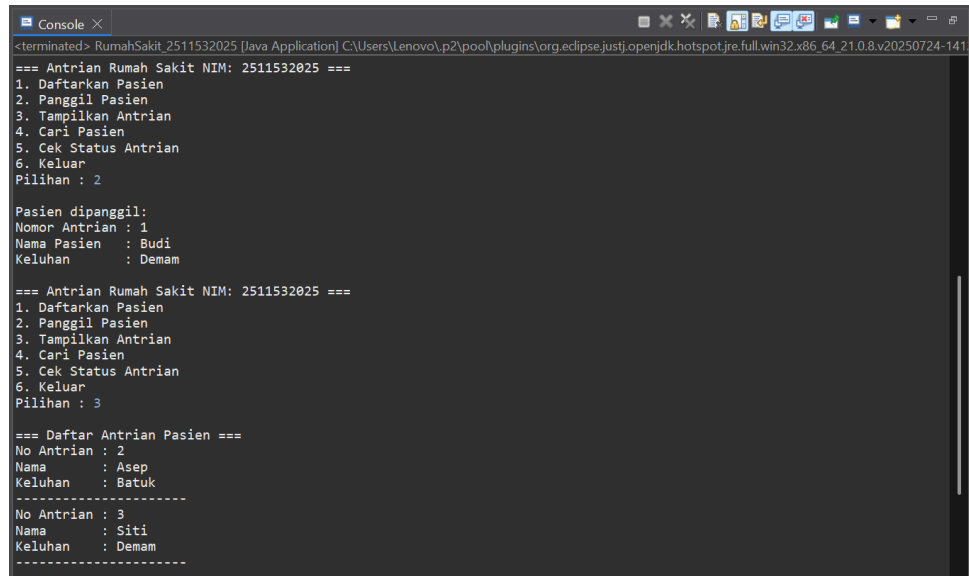
```

<terminated> RumahSakit_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-141
=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : Budi
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : Asep
Masukkan Keluhan : Batuk
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : Siti
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 3

```



```

<terminated> RumahSakit_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-141
=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 2

Pasien dipanggil:
Nomor Antrian : 1
Nama Pasien : Budi
Keluhan : Demam

=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 3

=== Daftar Antrian Pasien ===
No Antrian : 2
Nama : Asep
Keluhan : Batuk
-----
No Antrian : 3
Nama : Siti
Keluhan : Demam
-----

```

```
=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 4
Masukkan nama pasien yang dicari : Siti

Pasien ditemukan
Nomor Antrian : 3
Nama Pasien : Siti
Keluhan : Demam

=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 5

=== Status Antrian ===
Jumlah Pasien : 2
Pasien Terdepan : Asep

=== Antrian Rumah Sakit NIM: 2511532025 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 6
Program selesai
```